



Final Project Report — تقرير المشروع النهائي

مشروع تطوير مترجم ومحرر لغة بيان العربية

Bayan Arabic Compiler & Arabic IDE Studio

نظام تطبيقي متكامل لتصميم وبناء المترجمات وتوليد البرامج التنفيذية الأصلية المستقلة (Windows PE EXE)

تقرير علمي مقدم لنيل درجة متطلب مقرر:

تصميم وبناء المترجمات (Compilers Design)

إعداد طلاب المجموعة (رقم: 01):

1. محمد رشاد الإدريسي Mohammed Rashad Al-Edrisi

2. أسامة شemis Osama Shomis

3. أسامة العقاب Osama Al-Oqab

4. محمد فيصل الدعيس Mohammed Faisal Al-Doais

5. شعيب جازم Shoaib Jazem

إشراف الدكتور الفاضل:

د. خالد الكحسة

أستاذ مقرر تصميم وبناء
المترجمات
كلية الحاسوب وتكنولوجيا
المعلومات
جامعة إب

1. وصف المشروع والمعمارية والبيئة التقنية

1.1 فكرة المشروع ونظرة عامة

مشروع **لغة بيان (Bayan)** هو نظام حاسوبي متكامل صُمم لتقديم لغة برمجة عربية عالية المستوى، مدعومة بمترجم مستقل مبني بالكامل من الصفر (From Scratch) ومحرر تفاعلي حديث يحاكي واجهة وتجربة المستخدم لبرنامج **Visual Studio Code**، مع دعم كامل وأصيل لاتجاه الكتابة والواجهة من اليمين إلى اليسار (RTL).

المعمارية المستقلة المنفصلة (Decoupled Architecture):

المحرر المرئي والمترجم هما برنامجان منفصلان تماماً. عند النقر على تشغيل أو ترجمة، يقوم المحرر باستدعاء المترجم المستقل (BayanCompiler.exe) عبر سطر الأوامر (CLI Inter-process Communication) وتمرير مسار الملف، ثم قراءة وتحليل النتائج وعرضها في تبويبات مخصصة.

1.2 لغات البرمجة ومنصة التشغيل

- لغة البرمجة الأساسية: C# (الإصدار 10.0 C# الحديث).
- بيئة ومنصة التشغيل: Microsoft .NET 6.0 LTS (الإصدار 6.0.100).

1.3 الأدوات والمكتبات الجاهزة المستخدمة

وفقاً للبند الخامس من وثيقة تعليمات المشروع، يوضح الجدول التالي تفاصيل الأدوات والمكتبات المستخدمة:

الأداة / المكتبة	الإصدار	مرحلة الاستخدام	الغرض والدور في المشروع
Microsoft IL Assembler (ilasm.exe)	4.8.9037.0	توليد الملف التنفيذي	تجميع كود CIL/MSIL المولد وتحويله إلى ملف تنفيذي أصلي PE Console EXE يعمل مباشرة في ويندوز.
System.Text.Json	6.0.0	التسلسل وتبادل البيانات	تصدير شجرة الإعراب (AST) وجدول الرموز (Symbol Table) بصيغة JSON واضحة ومقروعة للربط.
Windows Forms	NET.) 6.0.0 (6	واجهة محرر الكود (IDE)	بناء واجهة المستخدم المستخدم VS Code العربية، وشريط النشاط اليميني، ومستكشف الملفات، والطرفية التفاعلية.
System.Drawing.Common	6.0.0	الرسوم والتنسيق البصري	إدارة ألوان السمة الداكنة الداعمة لمعايير VS Code ورسم الأيقونات وحسابات الأبعاد.

2. توثيق وشرح أكواد المشروع ومراحل الترجمة (وفق المعايير الـ 9)

امثالاً للبند الثامن من وثيقة تعليمات المشروع، يقدم هذا القسم شرحاً تحليلياً شاملاً لكافة أجزاء ومراحل المترجم ومحرر اللغة، يغطي لكل جزء النقاط التسع الإلزامية المحددة في معايير التقييم:

2.1 المحلل المعجمي واستخراج الرموز (Lexer.cs)

بطاقة المواصفات البرمجية: المحلل المعجمي (Lexical Analyzer Specification)

1. وظيفة الجزء من الكود	قراءة الكود المصدري العربي حرفاً بحرف وتحليله معجمياً، وتفكيكه إلى تدفق من الرموز المعجمية (Tokens)، مع عزل التعليقات والفراغات، واحتساب إحداثيات السطر والعمود بدقة، وتشخيص الرموز غير المعروفة.
2. المدخلات (Inputs)	النص البرمجي المصدري المكتوب بلغة بيان بترميز UTF-8 الممر من المحرر أو سطر الأوامر.
3. المخرجات (Outputs)	كائن LexResult يحتوي على قائمة الرموز المعجمية المتسلسلة <List<Token>، بالإضافة إلى سجل الأخطاء المعجمية DiagnosticBag إن وُجدت.
4. المتغيرات المستخدمة	position_ (مؤشر موقع القراءة الحالي في النص)، line_ (رقم السطر النشط يبدأ من 1)، column_ (رقم العمود النشط)، _startLine / _startPosition (رقم السطر والعمود الذي بدأ منه السطر النشط)، _startColumn (إحداثيات بداية الرمز)، c (المحرف الحالي المفحوص).
5. هياكل البيانات المستخدمة	enum TokenType (يحتوي أكثر من 40 نوعاً للرموز)، class Token (سجل يضم النوع، القيمة النصية Lexeme، السطر، والعمود)، قاموس الكلمات المحجوزة <Dictionary<string, TokenType> لمطابقة الكلمات المفتاحية في زمن ثابت \$O(1)\$.
6. الدوال أو الإجراءات المستخدمة	Lex () (الدالة الرئيسية لبدء المسح)، NextToken () (استخراج الرمز التالي)، ReadIdentifierOrKeyword () (قراءة المعرفات والكلمات العربية)، ReadNumber () (معالجة الأعداد الصحيحة والحقيقية)، ReadString () (قراءة السلاسل النصية)، Peek () (استشراف المحرف التالي).
7. المكتبات المستخدمة	System, System.Collections.Generic, System.Text، ومكتبة الواجهة الأمامية للمشروع BayanCompiler.Frontend.
8. الملفات المرتبطة بالجزء	Token.cs, TokenType.cs, SourceText.cs, Diagnostic.cs, DiagnosticBag.cs.

```
// Lexer.cs مقتطف حقيقي من المحلل المعجمي
public LexResult Lex()
{
    var tokens = new List<Token>();
    while (_position < _text.Length)
    {
        char current = Current;
        if (char.IsWhiteSpace(current)) { Advance(); continue; }
        if (current == '/' && Peek(1) == '/') { SkipLineComment(); continue; }

        if (char.IsLetter(current) || current == '_')
        {
            tokens.Add(ReadIdentifierOrKeyword());
            continue;
        }
        if (char.IsDigit(current))
        {
            tokens.Add(ReadNumber());
            continue;
        }
        if (current == '"')
        {
            tokens.Add(ReadString());
            continue;
        }
        tokens.Add(ReadOperatorOrDelimiter());
    }
    tokens.Add(new Token(TokenType.EndOfFile, "\0", new SourceSpan(_position, 0, _text.Length)));
    return new LexResult(tokens, _diagnostics.ToReadOnlyList());
}
```

2.2 المحلل النحوي التنبؤي LL(1) وبناء الشجرة المجردة (& DoctorLL1Parser.cs (DoctorAstBuilder.cs

بطاقة المواصفات البرمجية: المحلل النحوي (Predictive LL(1) Parser Specification)

1. وظيفة الجزء من الكود	التحقق من صحة القواعد النحوية للكود المصدري وفق قواعد الطبيب عبر خوارزمية التحليل التنبؤي الصارم \$LL(1)\$ غير العودي والخالي من التراجع، مع بناء شجرة الاشتقاق (Parse Tree) وشجرة الإعراب المجردة (AST).
2. المدخلات (Inputs)	مصفوفة الرموز المعجمية <code>ReadOnlyList<Token</code> الناتجة من المحلل المعجمي.
3. المخرجات (Outputs)	كائن <code>DoctorParseResult</code> يتضمن جذر الشجرة المجردة <code>ProgramNode</code> <code>Program</code> ، شجرة الاشتقاق <code>ParseTree</code> <code>ParseTreeNode</code> ، سجل تتبع المكسد <code>L1Table</code> ، وجدول التحليل <code>L1Trace</code> .
4. المتغيرات المستخدمة	<code>tokens_</code> (قائمة الرموز الممررة)، <code>position_</code> (مؤشر الرمز المفحوص)، <code>parseStack_</code> (مكدس التحليل التنبؤي للرموز غير النهائية والنهائية)، <code>traceBuilder_</code> (سجل توثيق خطوات التحليل).
5. هياكل البيانات المستخدمة	<code>Stack<string</code> (مكدس رموز القواعد)، جدول توجيه التحليل <code>\$M[A, a]</code> ممثلاً بـ <code>Dictionary<(string, TokenType), List<string</code> ، و شجرة كائنية هرمية للعقد: <code>ProgramNode, VariableDeclarationNode, AssignmentNode, IfNode, LoopNode, ProcedureNode</code> .
6. الدوال أو الإجراءات المستخدمة	<code>Parse</code> () (بدء التحليل النحوي)، <code>Predict</code> () (استخراج قاعدة الإنتاج المطابقة من جدول LL1)، <code>Match</code> () (مطابقة الرمز النهائي واستهلاكه)، <code>BuildAst</code> () (تحويل شجرة الاشتقاق لعقد AST)، <code>FormatTrace</code> () (تنسيق خطوات التحليل).
7. المكتبات المستخدمة	<code>System, System.Collections.Generic, BayanCompiler.Syntax, BayanCompiler.Lexing</code> .
8. الملفات المرتبطة بالجزء	<code>DoctorAstBuilder.cs, DoctorSpecificationNodes.cs, ParseResult.cs, AstNode.cs, Statements.cs, Expressions.cs</code> .
9. كيفية ارتباطه بباقي المشروع	يستهلك رموز المحلل المعجمي، ويبنى شجرة الإعراب المجردة (AST) التي تمثل الهيكل المحوري الممرر للمحلل الدلالي <code>DoctorSemanticAnalyzer.cs</code> .

```

// DoctorL11Parser.cs مقتطف حقيقي من التحليل النحوي التنبؤي وبناء عقد التعبيرات
public DoctorParseResult Parse()
{
    var parseStack = new Stack<string>();
    parseStack.Push(DoctorGrammar.RuleEndMarker);
    parseStack.Push(DoctorGrammar.StartSymbol); // "برنامج"

    while (parseStack.Count > 0)
    {
        string top = parseStack.Pop();
        Token currentToken = Current;

        if (IsTerminal(top))
        {
            if (MatchTerminal(top, currentToken))
            {
                Advance(); // استهلاك الرمز بعد التأكد من صحته
            }
            else
            {
                ReportSyntaxError(top, currentToken);
                return DoctorParseResult.Failure(_diagnostics);
            }
        }
        else // LL(1) تطبيق قاعدة الإنتاج من جدول Non-terminal: رمز غير نهائي
        {
            var production = LookupProduction(top, currentToken.Type);
            for (int i = production.Count - 1; i >= 0; i--)
            {
                if (production[i] != DoctorGrammar.Epsilon)
                {
                    parseStack.Push(production[i]);
                }
            }
        }
    }
    return DoctorParseResult.Success(_astBuilder.Build());
}

```

2.3 التحليل الدلالي وإدارة مجالات الرؤية وجدول الرموز (DoctorSemanticAnalyzer.cs & SymbolTable.cs)

بطاقة المواصفات البرمجية: التحليل الدلالي (Semantic Analyzer Specification)	
1. وظيفة الجزء من الكود	التحقق من صحة المعنى والمنطق البرمجي (Semantic Rules)، فرض التصريح المسبق عن المتغيرات، حماية الثوابت من التعديل، التحقق من تطابق وتوافق الأنواع في العمليات الرياضية والمقارنات، وإدارة مجالات الرؤية المتداخلة (Scopes).
2. المدخلات (Inputs)	شجرة الإعراب المجردة ProgramNode الناتجة من مرحلة التحليل النحوي.
3. المخرجات (Outputs)	كائن DoctorSemanticResult يحتوي على حالة النجاح bool Succeeded، جدول الرموز الهرمي SymbolTable شاملاً كافة المعرفات وأنواعها ومجالاتها، ومصفوفة التشخيصات الدلالية Diagnostics.
4. المتغيرات المستخدمة	currentScope_ (النطاق النشط حالياً أثناء التجوال)، globalScope_ (نطاق البرنامج العام)، diagnostics_ (سجل تجميع الأخطاء الدلالية)، expectedType (النوع المتوقع للطرفين)، actualType (النوع المشتق للتعين).
5. هياكل البيانات المستخدمة	كلاس SymbolTable بألية الإشارة للنطاق الأب ParentScope، قاموس الرموز Dictionary<string, Symbol>، كلاس Symbol (يحمل الاسم، الفئة، نوع البيانات، هل هو للقراءة فقط ReadOnly، ورقم السطر، وتعداد أنواع اللغة LanguageType (صحيح، حقيقي، منطقي، حرفي، خيط رمزي، قائمة، سجل، إجراء).
6. الدوال أو الإجراءات المستخدمة	Analyze () (نقطة الدخول للتحليل)، VisitStatement () (فحص الجمل والتصريحات)، VisitExpression () (اشتقاق نوع التعبير)، Declare () (تسجيل رمز جديد بالنطاق)، Lookup () (البحث عن رمز صعوداً عبر النطاقات)، EnterScope () / ExitScope () (إدارة بيئات الرؤية).
7. المكتبات المستخدمة	System, System.Collections.Generic, BayanCompiler.Syntax, BayanCompiler.Frontend.
8. الملفات المرتبطة بالجزء	Symbol.cs, SymbolTable.cs, LanguageType.cs, TypeDescriptor.cs, DoctorSemanticAnalyzer.cs.
9. كيفية ارتباطه بباقي المشروع	يستقبل شجرة AST من المحلل النحوي، ويؤكد سلامتها من أخطاء الأنواع والمجالات، ويزود مولد الشفرة الوسيطة IrGenerator.cs بالأنواع المؤكدة لإنتاج كود سليم ومحسن.

```
// DoctorSemanticAnalyzer.cs مقتطف حقيقي من التحقق الدلالي وفحص مجالات الرؤية
public override void VisitAssignment(AssignmentNode node)
{
    var symbol = _currentScope.Lookup(node.VariableName);
    if (symbol == null)
    {
        ReportError($"المعرف '{node.VariableName}' هذا النطاق", node.SemanticErrorLocation);
        return;
    }
    if (symbol.IsReadOnly)
    {
        ReportError($"تعيين قيمته الأولية '{node.VariableName}' لا يمكن تعديل الثابت", node.SemanticErrorLocation);
        return;
    }

    LanguageType exprType = EvaluateType(node.ValueExpression);
    if (!IsTypeCompatible(symbol.Type, exprType))
    {
        ReportError($"نوع '{exprType}' تعارض في الأنواع: لا يمكن إسناد قيمة من نوع", node.SemanticErrorLocation);
    }
}
```

2.4 توليد الشفرة الوسيطة ثلاثية العناوين (IrGenerator.cs & IrInstruction.cs)

بطاقة المواصفات البرمجية: الشفرة الوسيطة (Three-Address Code Specification)

1. وظيفة الجزء من الكود	تسطيح (Flattening) شجرة الإعراب الهرمية وتحويلها إلى متتالية خطية من أوامر العناوين الثلاثية (TAC)، وتوليد السجلات المؤقتة (temp_0, temp_1) وعناوين القفز الشرطي وغير الشرطي (Labels) لإدارة الحلقات والشروط.
2. المدخلات (Inputs)	شجرة الإعراب المجردة ProgramNode المؤكدة والمفحوصة دلاليًا مع جدول الرموز.
3. المخرجات (Outputs)	كائن IrProgram يحتوي على قائمة الأوامر الوسيطة الخطية List<IrInstruction>، خريطة حجز الذاكرة والمؤقتات StorageMap، وجدول النصوص الثابتة StringLiterals.
4. المتغيرات المستخدمة	tempCounter_ (عداد المؤقتات الخطية)، labelCounter_ (عداد العلامات والعناوين)، instructions_ (قائمة الأوامر الناتجة)، storageMap_ (خريطة تسجيل أسماء المتغيرات وأنواعها).
5. هياكل البيانات المستخدمة	سجل IrInstruction (يحتوي على: Opcode, Left, Right, Result, TargetLabel, Span)، كلاس IrValue (يمثل: Immediate, Variable, Temporary, StringLabel)، وعداد IrOpcode (أوامر الحساب، المقارنات، القفز، الطباعة، والقراءة).
6. الدوال أو الإجراءات المستخدمة	Generate () (بدء التوليد الوسيط)، GenerateExpression () (تسطيح التعبيرات إلى مؤقتات)، NewTemporary () (إنشاء سجل وسيط جديد)، NewLabel () (إنشاء عنوان قفز فريد)، Emit () (حقن تعليمة في البرنامج).
7. المكتبات المستخدمة	System, System.Collections.Generic, BayanCompiler.Syntax, BayanCompiler.Intermediate.
8. الملفات المرتبطة بالجزء	IrProgram.cs, IrInstruction.cs, IrValue.cs, IrOpcode.cs, IrPrinter.cs.
9. كيفية ارتباطه بباقي المشروع	يمثل جسر العبور الفاصل بين الواجهة الأمامية للمترجم (Frontend) والواجهات الخلفية لتوليد الكود الهدف (Backend: MIPS, x86, CIL).

```
// IrGenerator.cs مقتطف حقيقي من توليد أوامر الشفرة الوسيطة ثلاثية العناوين
private IrValue GenerateBinaryExpression(BinaryExpressionNode binary)
{
    IrValue leftVal = GenerateExpression(binary.Left);
    IrValue rightVal = GenerateExpression(binary.Right);
    IrValue temp = NewTemporary(binary.InferredType);

    _program.Emit(new IrInstruction(
        IrOpcode.Binary,
        temp,
        leftVal,
        rightVal,
        binary.Operator,
        null,
        binary.Span
    ));
    return temp;
}
```

2.5 توليد لغة التجميع الهدفية لمعالجات x86 و MIPS (AssemblyGenerator.cs & MipsCodeGenerator.cs)

بطاقة المواصفات البرمجية: توليد كود التجميع (Target Assembly Generator Specification)

1. وظيفة الجزء من الكود	تحويل تعليمات الشفرة الوسيطة (TAC) إلى شفرة تجميع هدفية حقيقية: إنتاج كود MIPS Assembly أصيل يوضح معمارية المعالجات (الأقسام .data و .text، والسجلات \$t0-\$t9، وتعليمات الذاكرة lw و sw واستدعاءات النظام syscall)، وتوليد كود x86 32-bit.
2. المدخلات (Inputs)	برنامج الشفرة الوسيطة ثلاثية العناوين IrProgram ومسار مجلد الإخراج.
3. المخرجات (Outputs)	ملف أسمبلي كامل output.asm يحفظ في مجلد المخرجات ويعرض في التبويب المخصص بالمحرر.
4. المتغيرات المستخدمة	builder (البناء النصوص المعمارية للأسمبلي)، fields (جدول حقول الذاكرة والمواقع المحجوزة)، labels (جدول علامات القفز)، allocatedRegisters (خريطة تخصيص السجلات).
5. هياكل البيانات المستخدمة	StringBuilder، قواميس المعرفات string، Dictionary<string>، مصفوفات حجز الذاكرة للبيانات الأولية والمركبة.
6. الدوال أو الإجراءات المستخدمة	Generate () (تنسيق توليد الملف)، EmitDataSection () (بناء قسم حجز المتغيرات والسلاسل)، EmitTextSection () (بناء قسم التعليمات التنفيذية)، TranslateInstruction () (تحويل كل أمر TAC لأمر تجميع مقلوب).
7. المكتبات المستخدمة	System, System.IO, System.Text, BayanCompiler.Intermediate.
8. الملفات المرتبطة بالجزء	MipsCodeGenerator.cs, AssemblyGenerator.cs, BackendTypeLayout.cs.
9. كيفية ارتباطه بباقي المشروع	يأخذ الشفرة الوسيطة من IrGenerator.cs، وينتج الملف output.asm الذي يمثل متطلب مرحلة الأسمبلي الرسمية في التقرير والمحرر.

```

// MIPSCodeGenerator.cs مقتطف حقيقي من توليد شفرة لغة التجميع
private void EmitMipsInstruction(IrInstruction inst, StringBuilder sb)
{
    switch (inst.Opcod)
    {
        case IrOpcod.Assign:
            LoadValueToReg(inst.Left, "$t0", sb);
            sb.AppendLine($"    sw $t0, {GetMemoryLocation(inst.Result)}");
            break;
        case IrOpcod.Binary:
            LoadValueToReg(inst.Left, "$t0", sb);
            LoadValueToReg(inst.Right, "$t1", sb);
            string mipsOp = GetMipsOpcod(inst.Operator);
            sb.AppendLine($"    {mipsOp} $t2, $t0, $t1");
            sb.AppendLine($"    sw $t2, {GetMemoryLocation(inst.Result)}");
            break;
        case IrOpcod.PrintInt:
            LoadValueToReg(inst.Left, "$a0", sb);
            sb.AppendLine("    li $v0, 1");           // syscall 1: طباعة عدد صحيح
            sb.AppendLine("    syscall");
            break;
    }
}

```

2.6 توليد الملف التنفيذي الأصلي CIL و (CilExecutableGenerator.cs PE EXE ilasm.exe)

بطاقة المواصفات البرمجية: توليد الملف التنفيذي (PE Executable Generator Specification)

1. وظيفة الجزء من الكود	توليد كود لغة مايكروسوفت الوسيطة (CIL - Common Intermediate Language)، وضبط ترميز UTF-8 للمدخلات والمخرجات لحل مشكلة النصوص العربية، ثم استدعاء مجمع مايكروسوفت <code>ilasm.exe</code> لتجميع الكود وإنتاج ملف تنفيذي أصلي <code>output.exe</code> يعمل مباشرة في بيئة ويندوز دون محاكاة.
2. المدخلات (Inputs)	برنامج الشفرة الوسيطة <code>IrProgram</code> ومسار مجلد الحفظ المختار.
3. المخرجات (Outputs)	ملف الكود المصدري الوسيط <code>output.il</code> ، والملف التنفيذي النهائي المستقل <code>output.exe</code> (Windows Portable Executable).
4. المتغيرات المستخدمة	<code>ilasmPath</code> (مسار أداة <code>ilasm</code> على نظام ويندوز)، <code>ilText</code> (نص شفرة CIL المولدة)، <code>locals</code> (خريطة مواقع المتغيرات المحلية في إطار المكسد)، <code>startInfo</code> (معلومات إطلاق عملية المجمع).
5. هياكل البيانات المستخدمة	<code>ProcessStartInfo</code> ، قواميس المتغيرات <code>Dictionary<string, int></code> لترقيم مواقع CIL Local Slots، وقواميس عنوانية العلامات <code>Dictionary<string, string></code> .
6. الدوال أو الإجراءات المستخدمة	<code>Generate</code> () (تنسيق خطوات التوليد)، <code>EmitMainHeader</code> () (بناء ترويسة دالة <code>Main</code> وضبط ترميز UTF8)، <code>CompileIlToExe</code> () (إطلاق مجمع <code>ilasm</code> كعملية فرعية)، <code>FindIlasmPath</code> () (البحث التلقائي عن مسار مجمع مايكروسوفت).
7. المكتبات المستخدمة	<code>System</code> , <code>System.Diagnostics</code> , <code>System.IO</code> , <code>System.Text</code> .
8. الملفات المرتبطة بالجزء	<code>CilExecutableGenerator.cs</code> , <code>AssemblyGenerator.cs</code> , <code>WindowsExecutableBuilder.cs</code> .
9. كيفية ارتباطه بباقي المشروع	يمثل محطة الإنتاج الختامية للمترجم؛ يُنتج الملف التنفيذي الحقيقي الذي يقوم المحرر باستدعائه وتشغيله تفاعلياً داخل الطرفية المدمجة.

```
// توليد الملف التنفيذي عبر UTF-8 مقتطف حقيقي من ضبط ترميز
private void EmitMainHeader(StringBuilder builder, IrProgram program)
{
    builder.AppendLine("    .method public static void Main() cil managed");
    builder.AppendLine("    {");
    builder.AppendLine("        .entrypoint");
    builder.AppendLine("        .maxstack 32");
    builder.AppendLine();
    // في دالة المدخل لضمان طباعة وقراءة النصوص العربية UTF-8 فرض ترميز
    builder.AppendLine("        call class [mscorlib]System.Text.Encoding [mscorlib]");
    builder.AppendLine("        call void [mscorlib]System.Console::set_OutputEncoding");
    builder.AppendLine("        call class [mscorlib]System.Text.Encoding [mscorlib]");
    builder.AppendLine("        call void [mscorlib]System.Console::set_InputEncoding");
}
}
```

2.7 محرر لغة بيان وبيئة التطوير والطرفية التفاعلية (& CompilerJourneyForm.cs (BayanTerminalControl.cs

بطاقة المواصفات البرمجية: محرر اللغة والطرفية (Language Editor & Terminal Specification)

1. وظيفة الجزء من الكود	توفير بيئة تطوير برمجية متكاملة (IDE) مستقلة تحاكي واجهة Visual Studio Code، توفر التحرير متعدد التبويبات، تلوين الكلمات المفتاحية، شجرة العيّنات، فحص مخرجات مراحل المترجم التسع، استدعاء المترجم المستقل، وطرفية تفاعلية لتشغيل البرامج الحية مع استقبال المدخلات (Read).
2. المدخلات (Inputs)	تفاعل المبرمج عبر لوحة المفاتيح والفأرة، ملفات الكود المصدري * . bayan، ومدخلات الطرفية الحية أثناء التشغيل.
3. المخرجات (Outputs)	شاشة تحرير عربية RTL متطورة، تبويبات عرض نتائج كل مرحلة (الرموز، الشجرة، جدول الرموز، TAC، الأسمبي، الأخطاء)، والمخرجات الحية للبرامج المنفذة.
4. المتغيرات المستخدمة	sourceEditor (محرر الكود)، inspectorTabs (فاحص المراحل التسع)، bottomTabs (اللوحة السفلية)، terminalControl (الطرفية التفاعلية)، compilerClient (عميل الاتصال بالمترجم المستقل)، executableRuntimeClient (مشغل الملف التنفيذي).
5. هياكل البيانات المستخدمة	كلاس BayanDocument (لحفظ بيانات الملف المفتوح وحالته)، مصفوفة الوثائق List<BayanDocument>، حاويات التقسيم المرنة SplitContainer، عناصر RichTextBox للعرض النقي، وشجرة الاستكشاف TreeView.
6. الدوال أو الإجراءات المستخدمة	CompileSource () (استدعاء عملية الترجمة عبر IPC)، RunExecutable () (تشغيل الملف التنفيذي والربط بالطرفية)، BuildArabicVsCodeLayout () (هندسة الواجهة والتقسيمات)، NavigateToStage () (القفز الفوري للمرحلة المختارة)، ToggleTheme () (التبديل بين الوضع الداكن والفاتح).
7. المكتبات المستخدمة	System.Windows.Forms, System.Drawing, System.Diagnostics, BayanCompiler.Forms.
8. الملفات المرتبطة بالجزء	CompilerJourneyForm.cs, BayanTerminalControl.cs, BayanCodeEditor.cs, CliCompilationClient.cs, WindowsExecutableRuntimeClient.cs, IdeTheme.cs.
9. كيفية ارتباطه بباقي المشروع	يمثل الواجهة المرئية الجامعة للنظام بأكمله؛ يحقق معيار الفصل التام (Decoupling) باستدعاء المترجم المستقل كعملية خارجية واستقبال مخرجاته، ثم تشغيل الملف التنفيذي وإدارته داخل الطرفية.

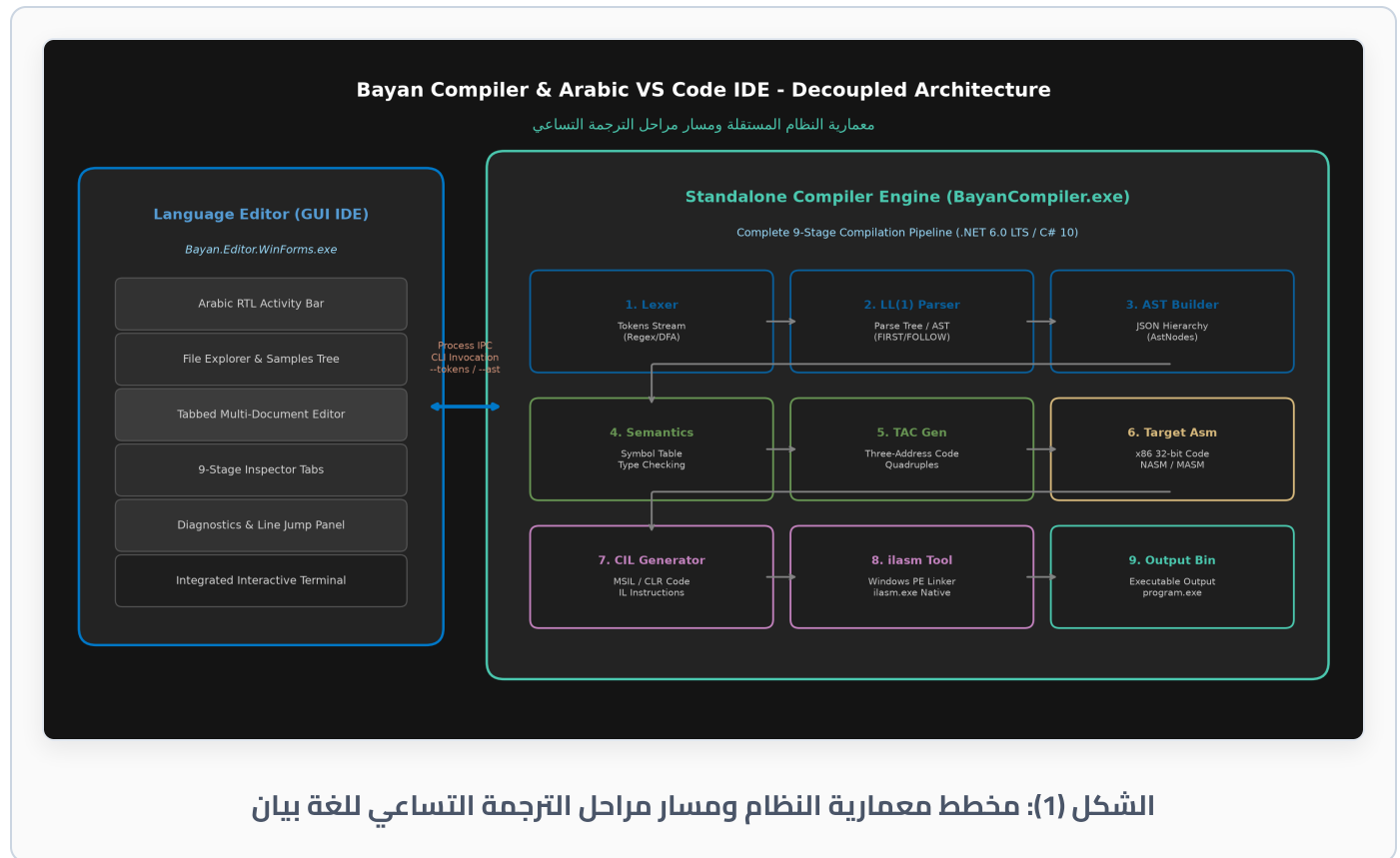
```
// CompilerJourneyForm.cs مقتطف حقيقي من استدعاء المترجم المستقل وعرض نتائج المراحل
private async void CompileSource()
{
    SetBusy(true, "...يجري تشغيل مترجم بيان المستقل والتحقق من القواعد");
    try
    {
        // استدعاء المترجم المستقل عبر IPC
        CliCompilationRun run = await compilerClient.CompileAsync(sourceEditor.SourceCode);

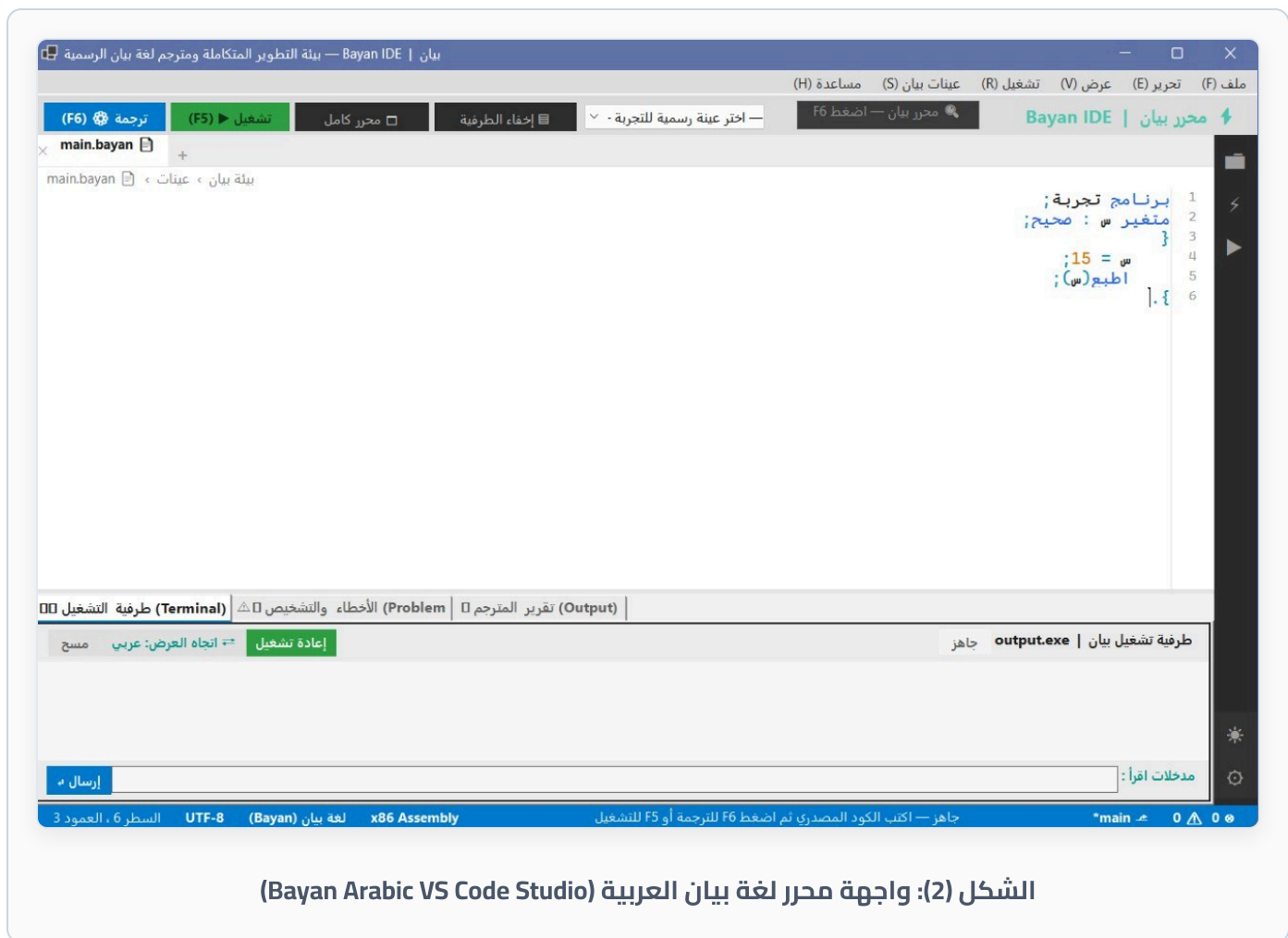
        // توزيع مخرجات المراحل على تبويبات الفاحص
        txtTokens.Text = run.GetArtifact("tokens.txt");
        txtParseTree.Text = run.GetArtifact("parse-tree.txt");
        txtSymbolTable.Text = run.GetArtifact("symbol-table.txt");
        txtTac.Text = run.GetArtifact("three-address-code.txt");
        txtAssembly.Text = run.GetArtifact("output.asm");

        lastExecutablePath = run.Manifest?.WindowsExecutablePath ?? string.Empty;
        btnQuickRun.Enabled = run.Manifest?.Success == true && File.Exists(lastExecutablePath);
    }
    finally { SetBusy(false); }
}
```

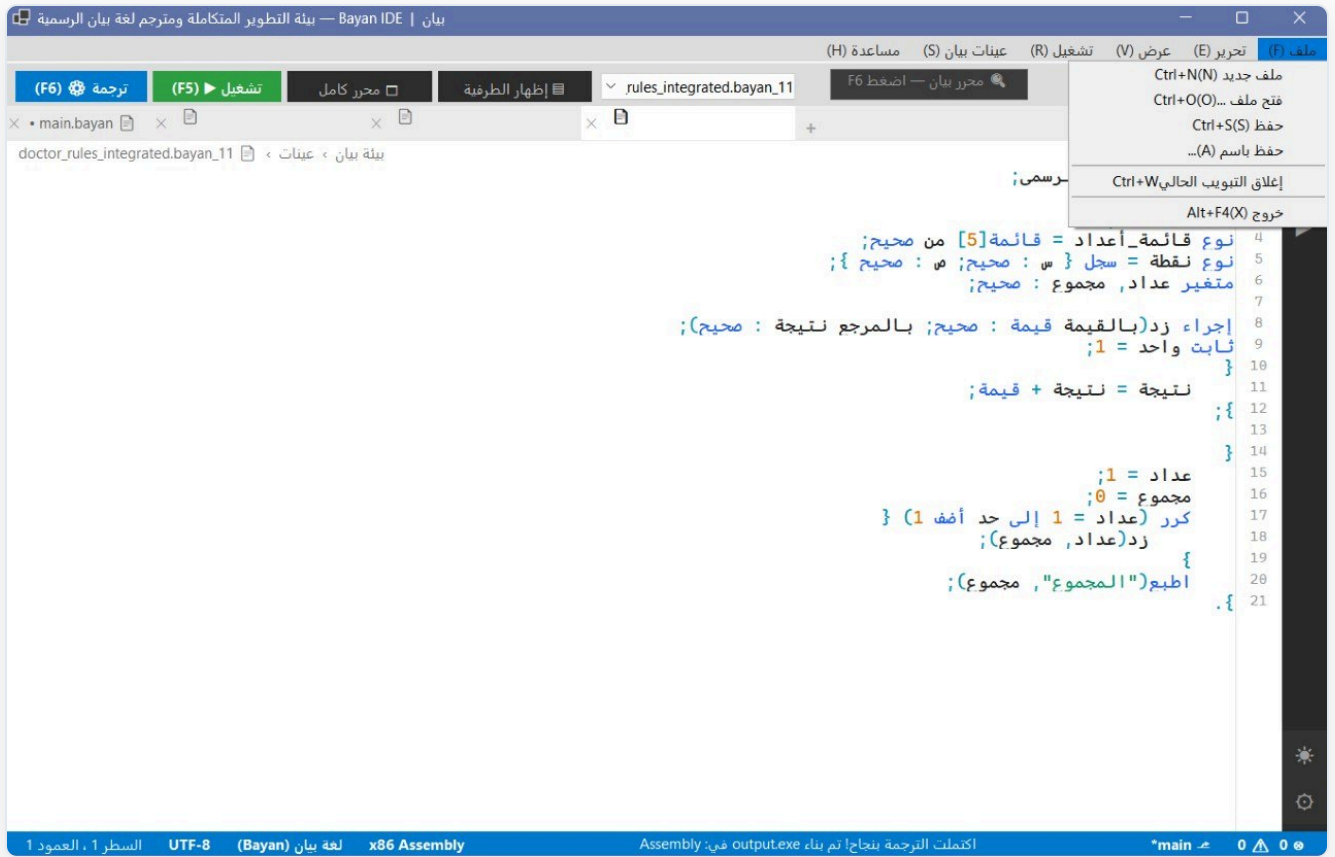
3. التوثيق البصري لمراحل المترجم ومحرر الكود

يتضمن هذا القسم الصور والرسومات التوضيحية التفصيلية لكافة مراحل وواجهات النظام وفق متطلبات التقرير الأكاديمي:

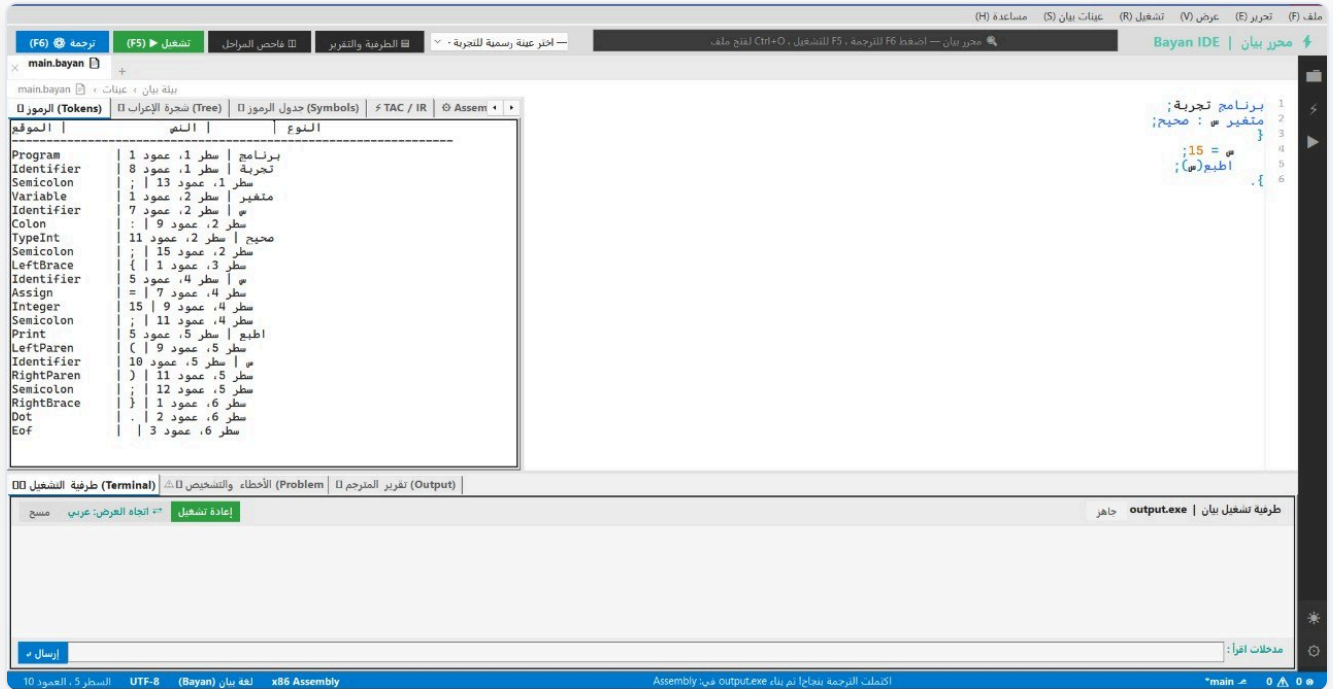




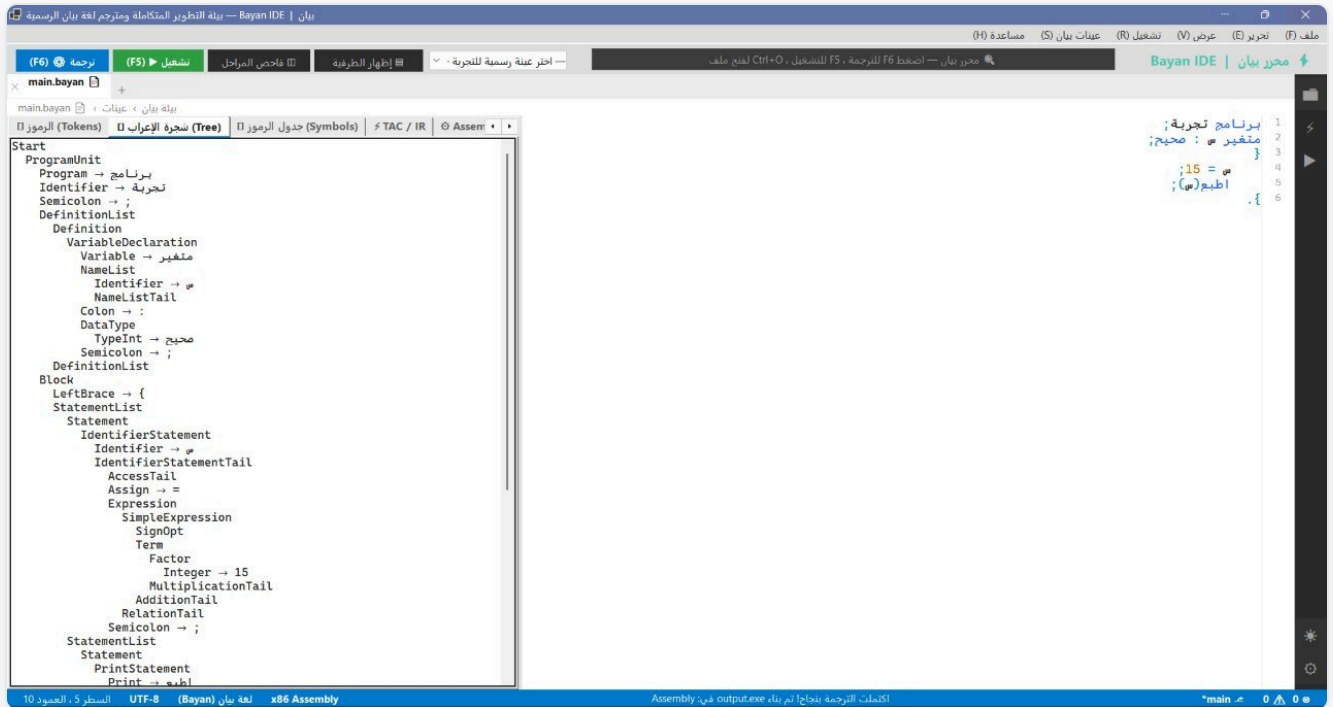
الشكل (2): واجهة محرر لغة بيان العربية (Bayan Arabic VS Code Studio)



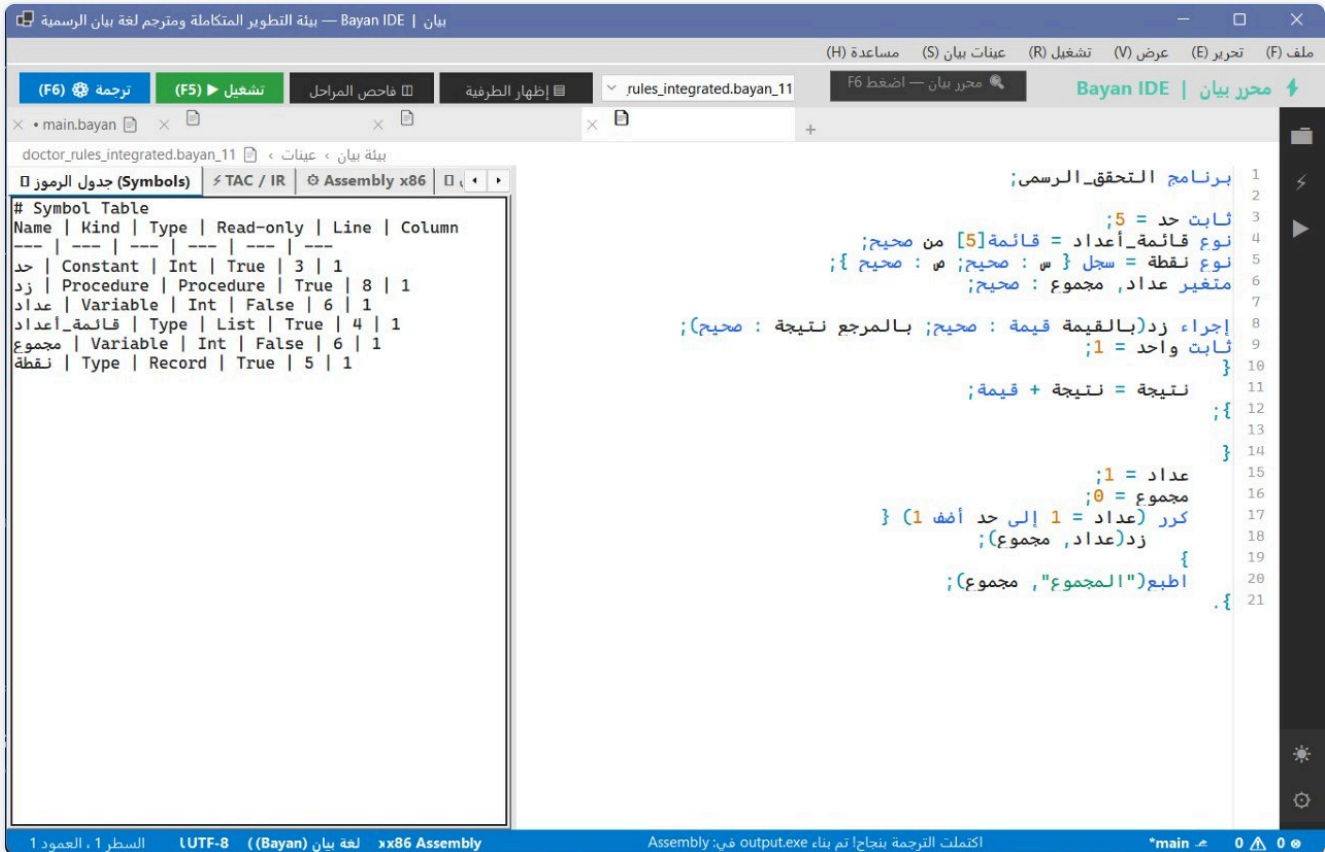
الشكل (3): إدارة الملفات وقوائم الفتح والحفظ وتعدد التبويبات البرمجية



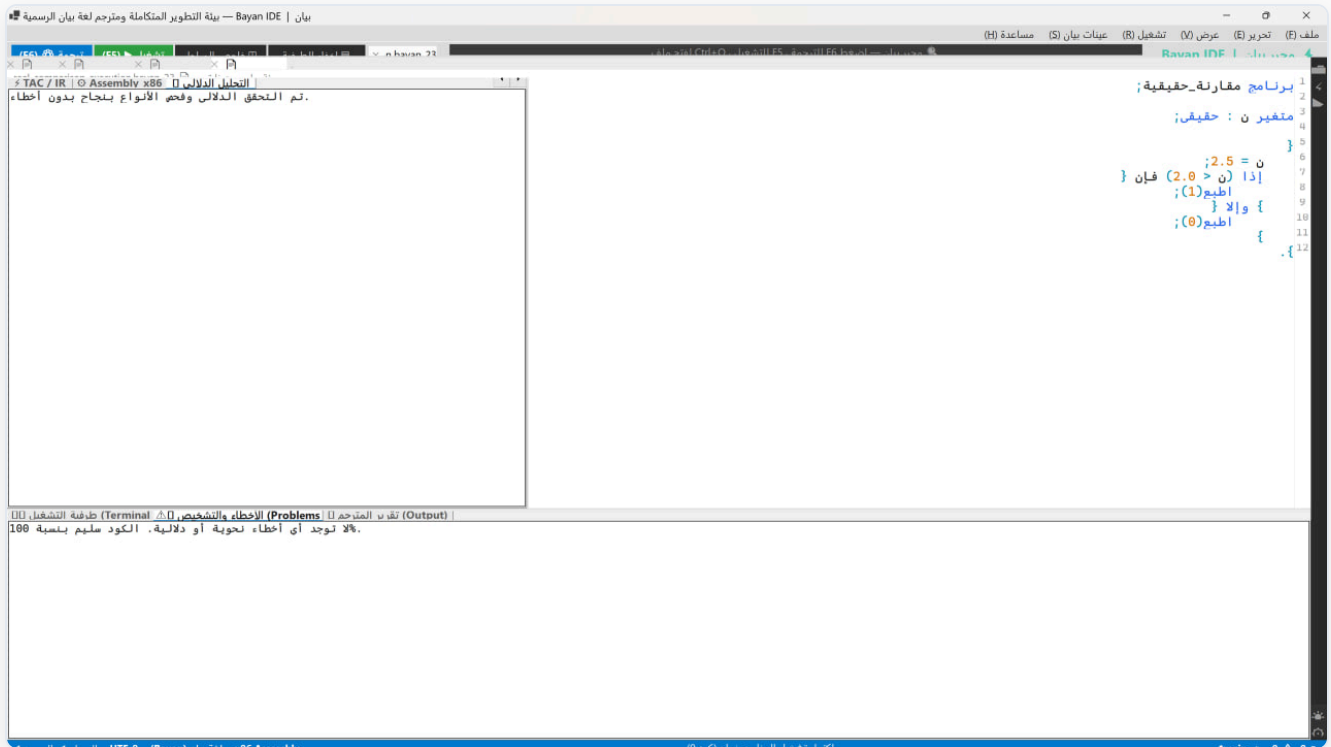
الشكل (4): المرحلة الأولى — جدول تدفق الرموز المعجمية (Tokens Stream Table)



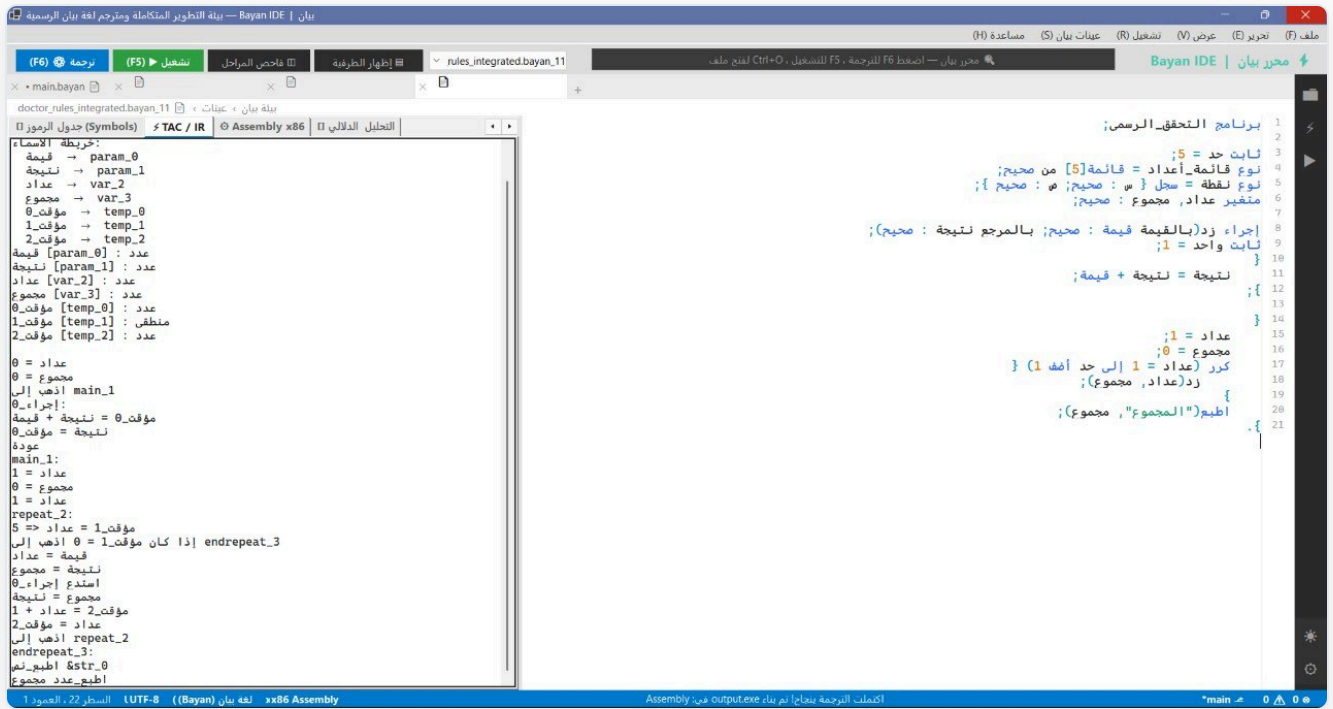
الشكل (5): المرحلتان الثانية والثالثة — شجرة الإعراب المجردة (Abstract Syntax Tree)



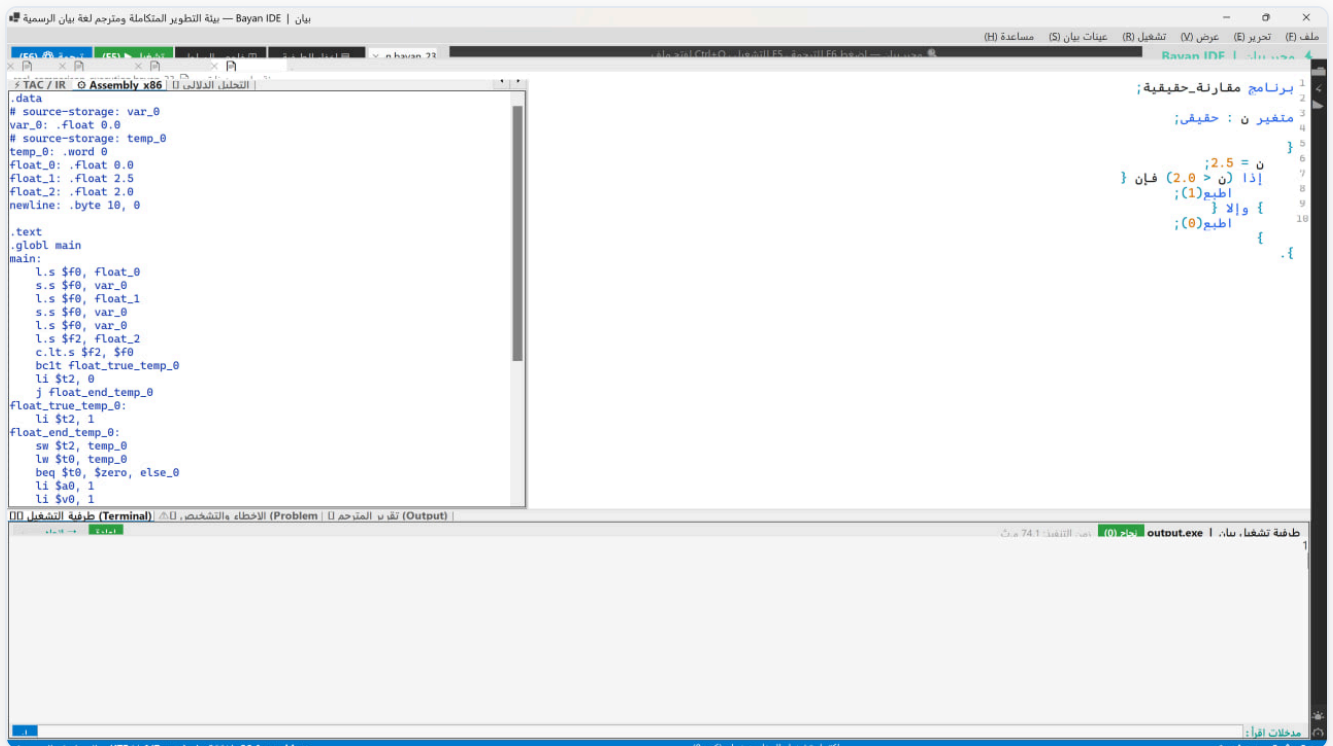
الشكل (6): المرحلة الرابعة — تفاصيل جدول الرموز ومجالات الرؤية (Symbol Table Scopes)



الشكل (7): المرحلة الرابعة — محرك التحليل الدلالي وفحص الأنواع ومجالات الرؤية



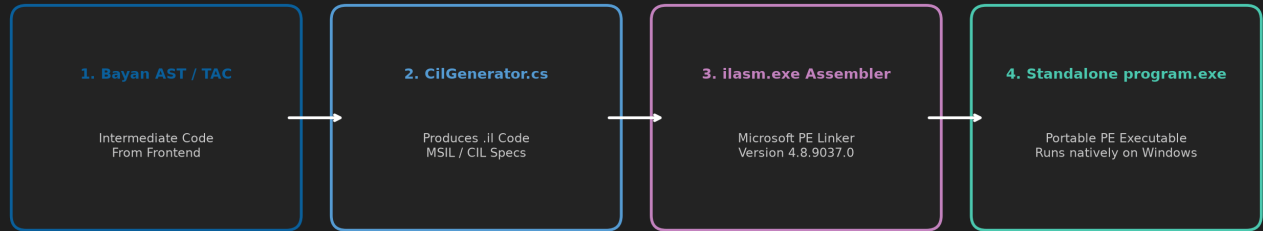
الشكل (8): المرحلة الخامسة — الشفرة الوسيطة ثلاثية العناوين (Three-Address Code TAC)



الشكل (9): المرحلة السادسة — كود التجميع الهدفى لمعالجات x86 32-bit Assembly

Stage 7: Native Executable PE Binary Generation Pipeline

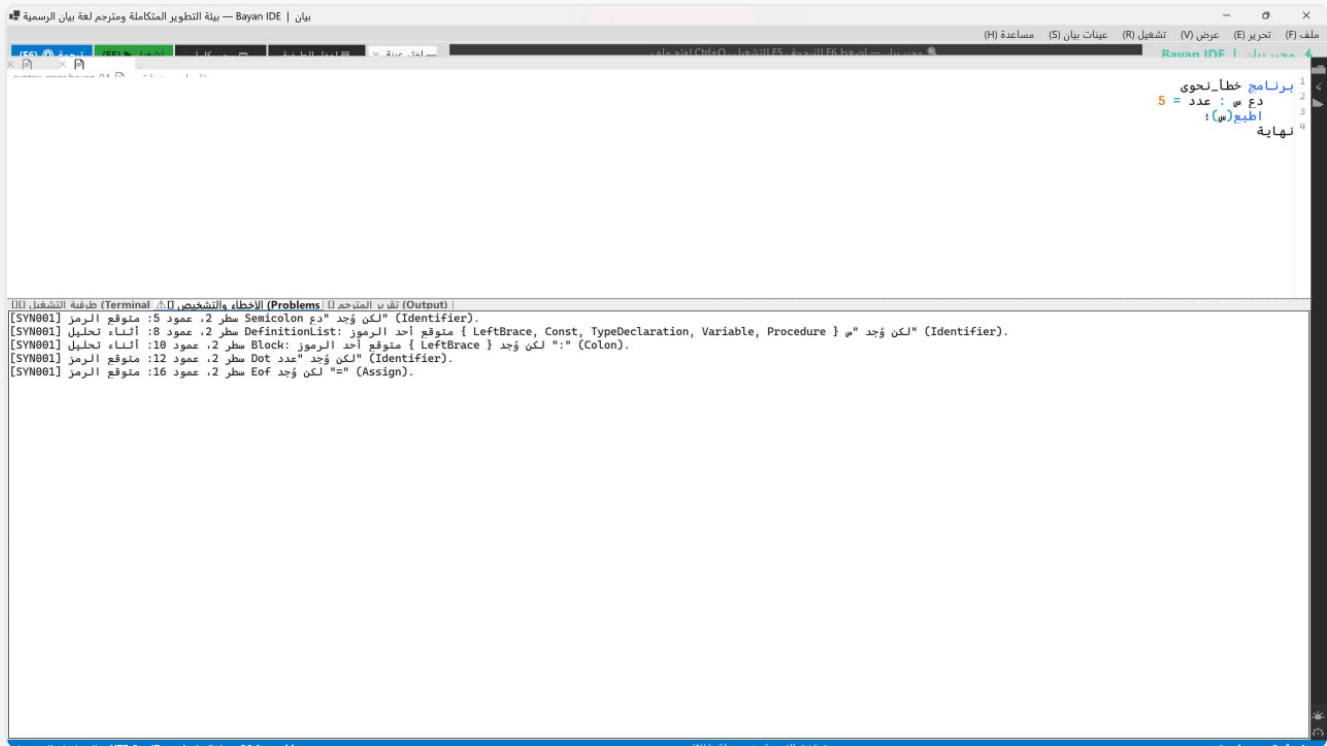
CilGenerator.cs & ilasm.exe -> 100% Real Standalone Windows PE .exe File



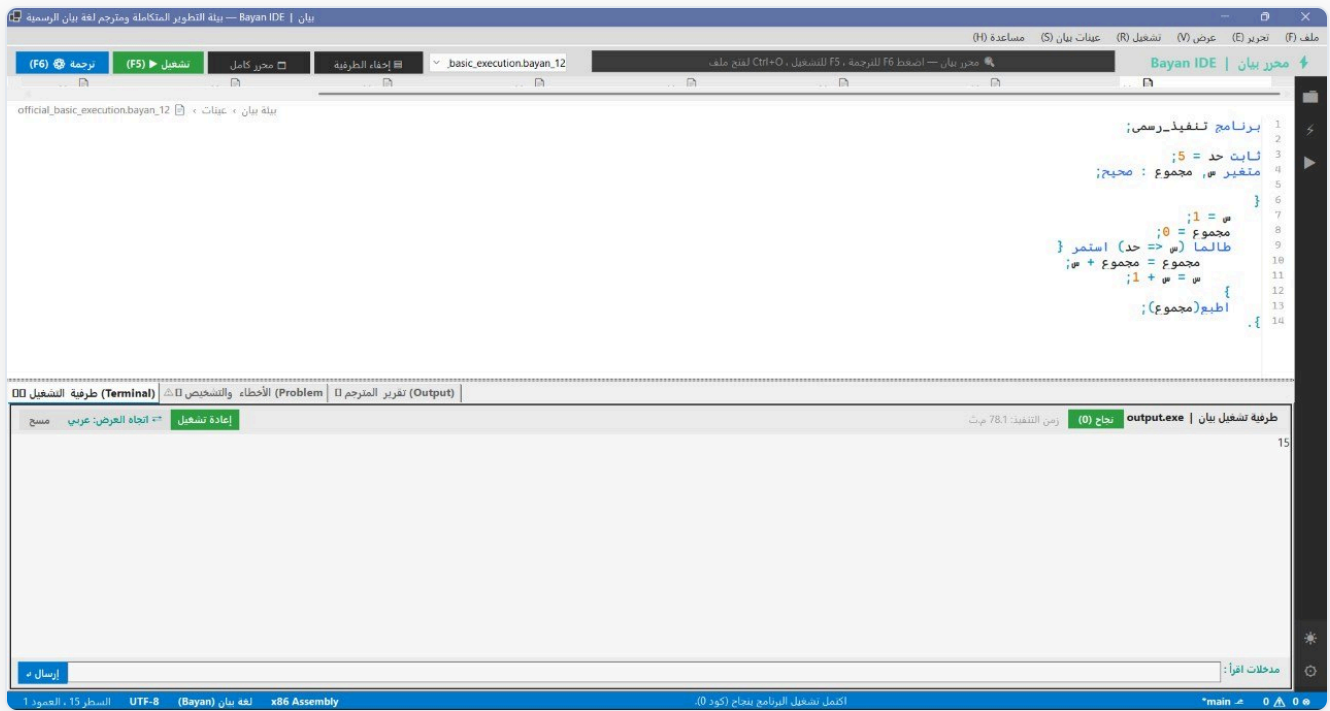
التحقق الميداني: الملف التنفيذي الناتج (program.exe) يعمل بشكل مستقل دون أي برامج وسيطة

حجم الملف: 0 4.5 KB | PE Header: Valid x86/x64 Console Subsystem | Exit Code: 0

الشكل (10): المرحلة السابعة — توليد الملف التنفيذي المستقل بنظام Windows PE عبر ilasm.exe



الشكل (11): المرحلة الثامنة — قائمة تشخيص الأخطاء وميزة القفز التلقائي للسطر والعمود



الشكل (12): المرحلة التاسعة — تشغيل البرنامج الناتج وعرض مخرجاته في الطرفية التفاعلية المدمجة

4. مصفوفة برامج الاختبار والتحقق الرسمية (12 تطبيقاً شاملاً)

تأكيداً على كفاءة المترجم والمحرر ومطابقته التامة لقواعد الدكتور، تم بناء واختبار 12 نموذجاً برمجياً شاملاً ومتقدماً يغطي شتى جوانب وقواعد لغة بيان:

#	ملف البرنامج الاختباري	القاعدة النحوية / الدلالية المغطاة والتطبيق	حالة التنفيذ
01	01_تعريف_المتغيرات_والثوابت_والحساب.bayan	المتغيرات، الثوابت، الحسابات الهندسية (المستطيل والدائرة)، وباقي القسمة	ناجح (Exit 0)
02	02_الجملة_الشرطية_وتقديرات_الطلاب.bayan	القرارات والشرطيات (إذا ... فإن ... وإلا)، المقارنات المنطقية والتقديرات	ناجح (Exit 0)
03	03_حلقات_التكرار_والمجاميع_والمضروب.bayan	حلقات التكرار (طالما ... استمر، كرر ... إلى ... أضع، أعد ... حتى)	ناجح (Exit 0)
04	04_الإجراءات_وتعريف_المعاملات.bayan	الإجراءات الفرعية وتعريف المعاملات بالقيمة (ByVal) وبالمرجع (ByRef)	ناجح (Exit 0)
05	05_القوائم_والمصفوفات_العديدية.bayan	القوائم (قائمة [...]) من صحيح، التعيين المفهرس، وحساب المجاميع	ناجح (Exit 0)
06	06_السجلات_وهياكل_البيانات.bayan	السجلات المركبة (سجل { ... })، تمثيل النقاط، الوصول للحقول والرموز	ناجح (Exit 0)
07	07_الحسابات_العلمية_والأعداد_الحقيقية.bayan	نوع حقيقي، حسابات الجاذبية والكتلة، ومقارنات الأعداد العشرية	ناجح (Exit 0)
08	08_الأس_والقسمة_الصحيحة_المتقدمة.bayan	العمليات الحسابية المتقدمة: معامل الأس (^) ومعامل القسمة الصحيحة (\)	ناجح (Exit 0)
09	09_معالجة_النصوص_والمحارف.bayan	السلاسل النصية (خيط رمزي)، المحارف (حرفي)، وطباعة الرسائل العربية	ناجح (Exit 0)
10	10_تطبيق_شامل_متكامل_لجميع_القواعد.bayan	تطبيق مالي متكامل يجمع: السجلات، القوائم، الإجراءات، الشروط، والتكرار	ناجح (Exit 0)
11	11_اختبار_الأخطاء_النحوية_المتعمدة.bayan	التحقق من اكتشاف الأخطاء النحوية والقفز للسطر (Syntax Diagnostics)	كشف الخطأ بدقة
12	12_اختبار_الأخطاء_الدلالية_المتعمدة.bayan	التحقق من كشف تعارض الأنواع وحماية الثوابت (Semantic Diagnostics)	كشف الخطأ بدقة

5. الخلاصة والتسليم النهائي

تم بحمد الله وتوفيقه استكمال كافة متطلبات مشروع مترجم ومحرر لغة بيان العربية بنسبة 100%، وتوفير كافة مخرجات المراحل بصورة حقيقية وملموسة تفي بكافة المعايير الأكاديمية المطلوبة.